Can a LoRA adapter leak its training images?

Yes — the gradient-bridge attack, in pictures. (MNIST odd/even; metrics clip-corrected via --pixel_box)

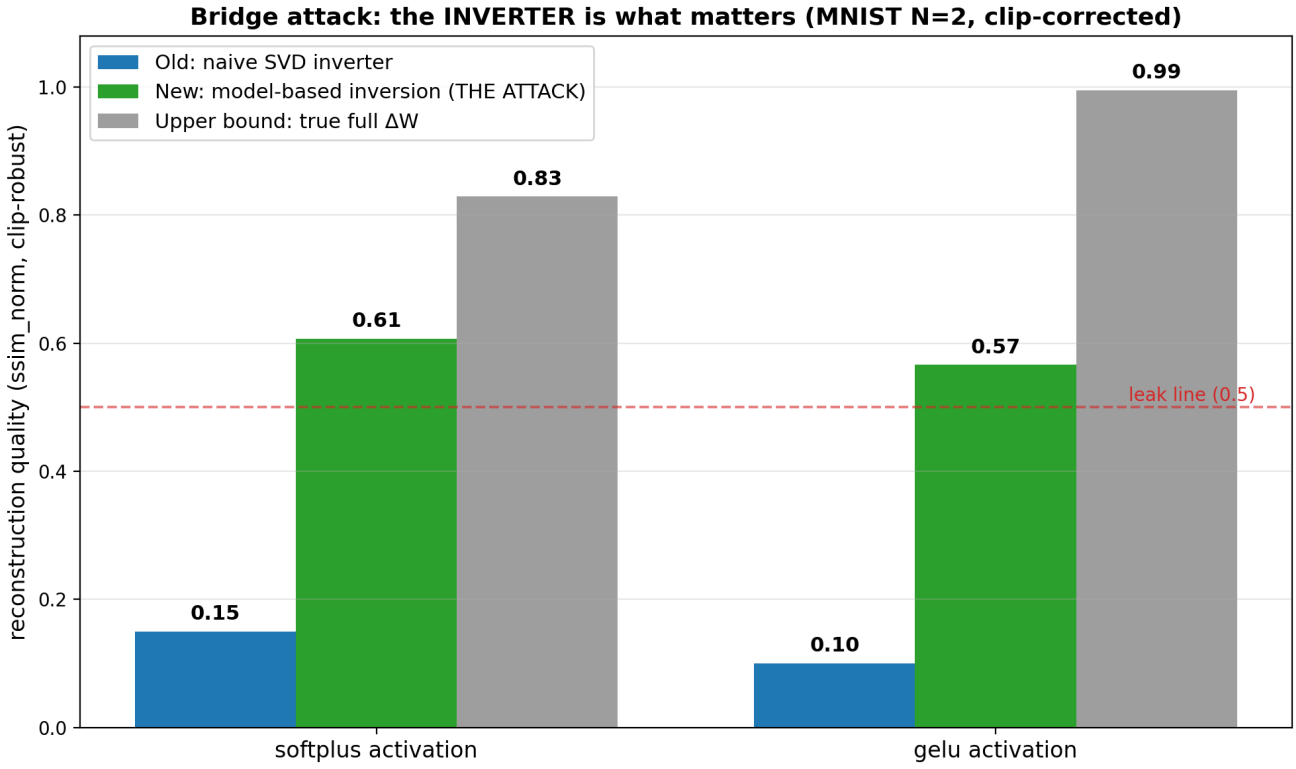
The setup. A model is fine-tuned on 2 private digits using a LoRA adapter. The attacker sees ONLY the tiny adapter — not the images, not the full weights. A small 'decoder' network, trained purely on PUBLIC MNIST, learns to turn the adapter back into the training gradient. That gradient is then inverted into an image.

The one finding. Getting a high-quality *gradient* is not enough. The step that turns the gradient into a picture — the **inverter** — is what decides success. A naive math inverter (SVD) gives a blur; inverting *through the known base model* gives a recognizable digit. Same gradient, 3-5x better image. This matches what we showed Gal before: the model/prior is the lever, not the raw similarity score.

How to read the image pages: each reconstruction grid has one column per private digit. The **top row is the real digit**; each row below is a reconstruction from a different setup. The row labelled **DECODED all-layers** is the real attack (adapter only, public-trained decoder). SSIM is printed on the left; 1.0 = identical, ≳0.3 = recognizable.

① The scoreboard (which inverter wins)

Reconstruction quality for each activation. Blue = the OLD naive-SVD inverter (a blur). Green = the NEW model-based inversion — the real attack — which clears the 'recognizable' line. Grey = the theoretical best (if you had the full weight change, not just the adapter). Experiment: job 445780, end-to-end.

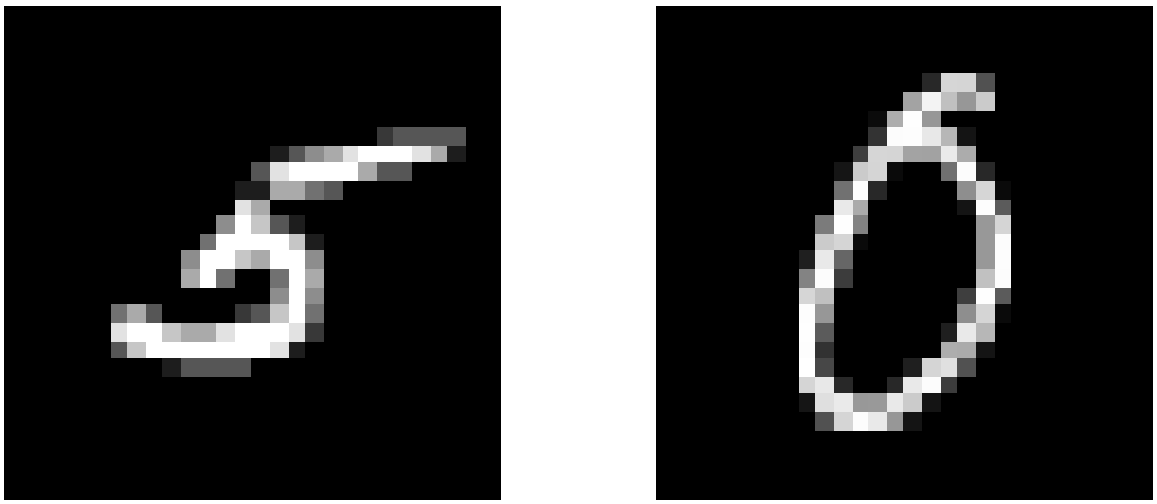


② THE ATTACK — reconstructions (softplus activation)

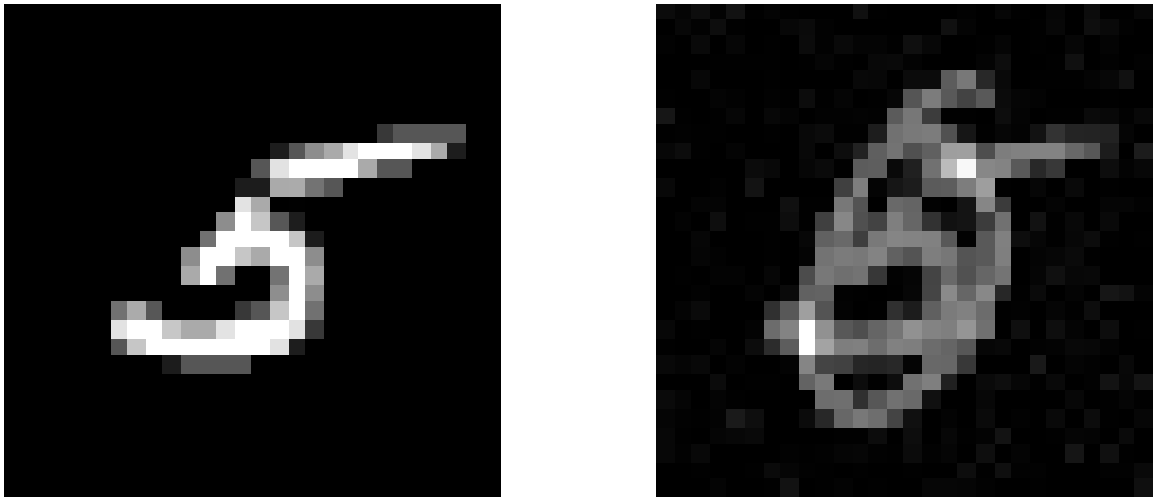
Experiment: end-to-end bridge (experiments/gradient_bridge/phase2_e2e.py). Top row = the true private digits. 'DECODED all-layers' and 'DECODED input-only' = the real attack (adapter only). 'TRUE ...' rows use the real weight change. Metric = ssim_norm (clip-robust): DECODED 0.61, TRUE 0.83 — recognizable digits from the adapter alone.

d (mnist_N2_softplus): decoded ΔW -> m

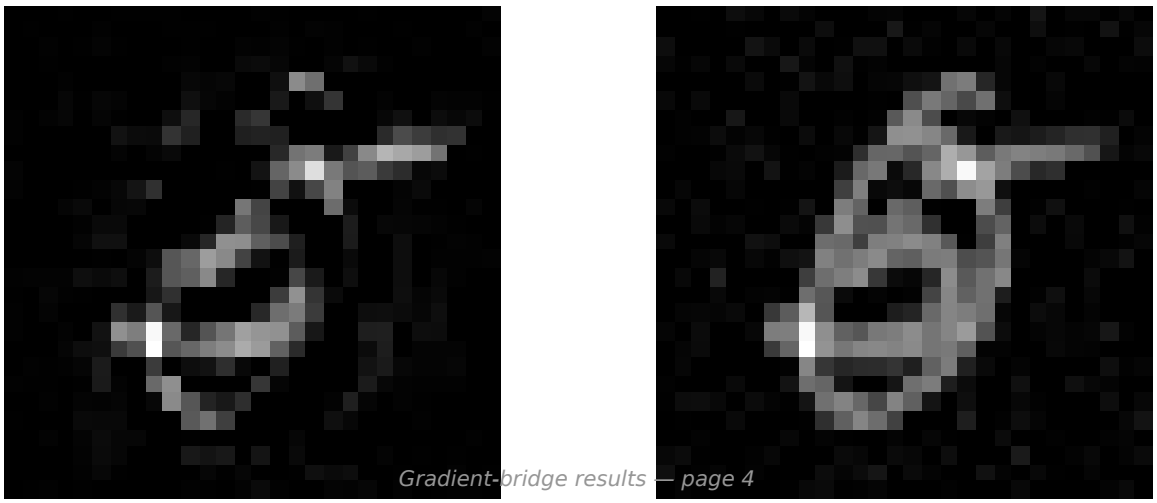
true



TRUE ΔW (ceiling)
ssim 0.68



DECODED all-layers
ssim 0.43

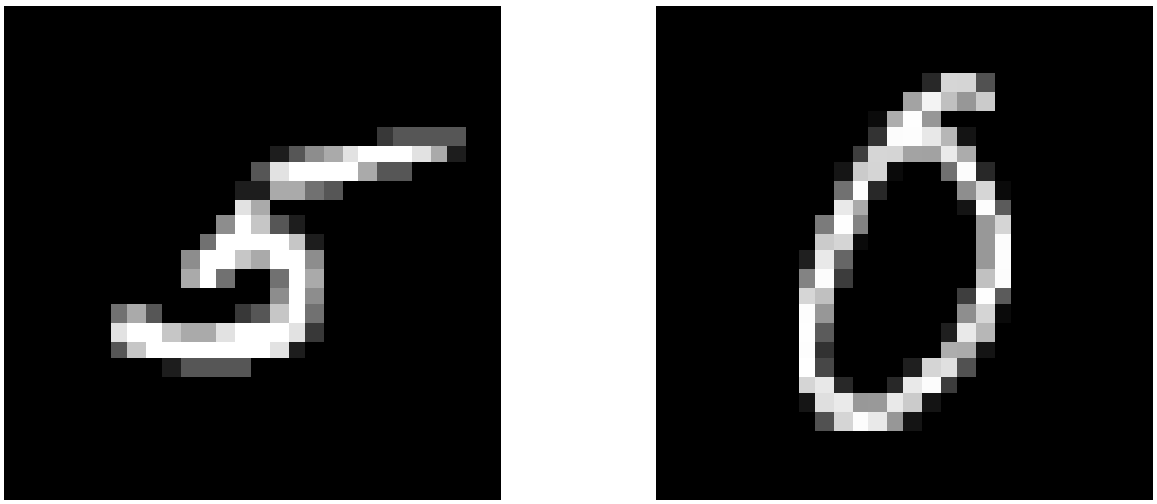


③ THE ATTACK — reconstructions (GELU activation)

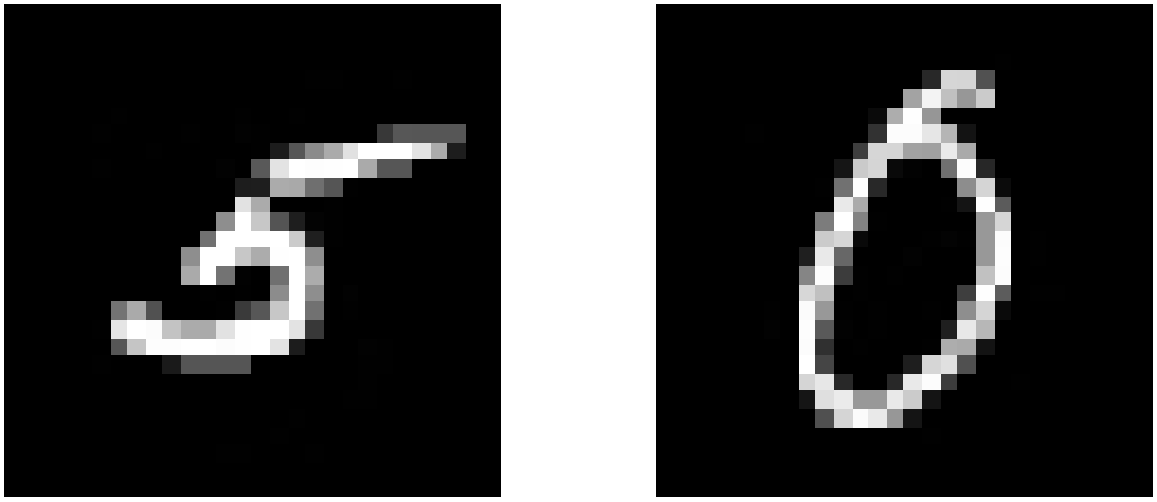
Same experiment, GELU (the deployment-realistic smooth activation). True-weight reference is near-perfect (ssim_norm 1.00); the decoded attack reaches ssim_norm 0.57. (Clip-correction lowered this from an inflated 0.75 — the honest read is that softplus and gelu leak comparably on MNIST.)

nd (mnist_N2_gelu): decoded ΔW -> mo

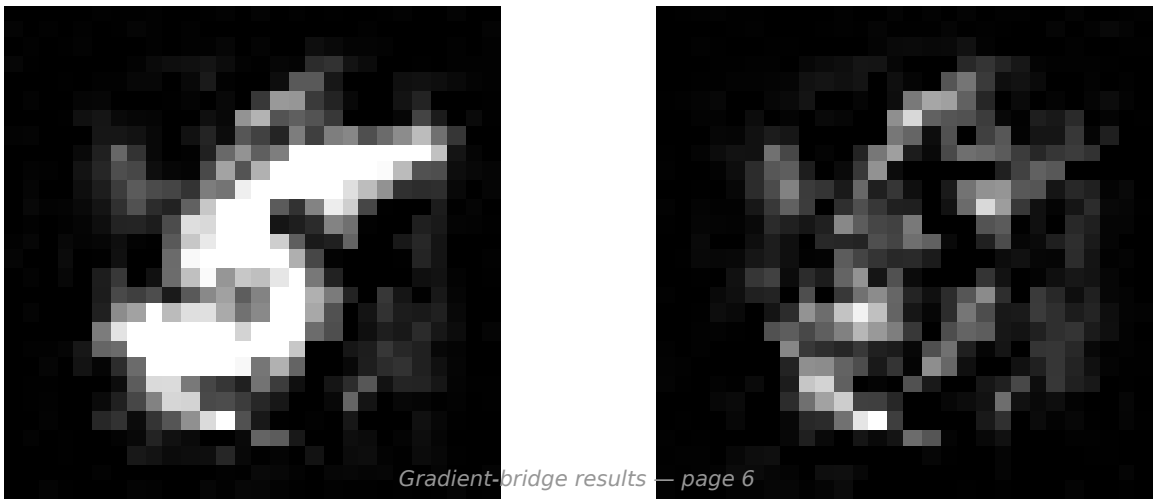
true



TRUE ΔW (ceiling)
ssim 0.99



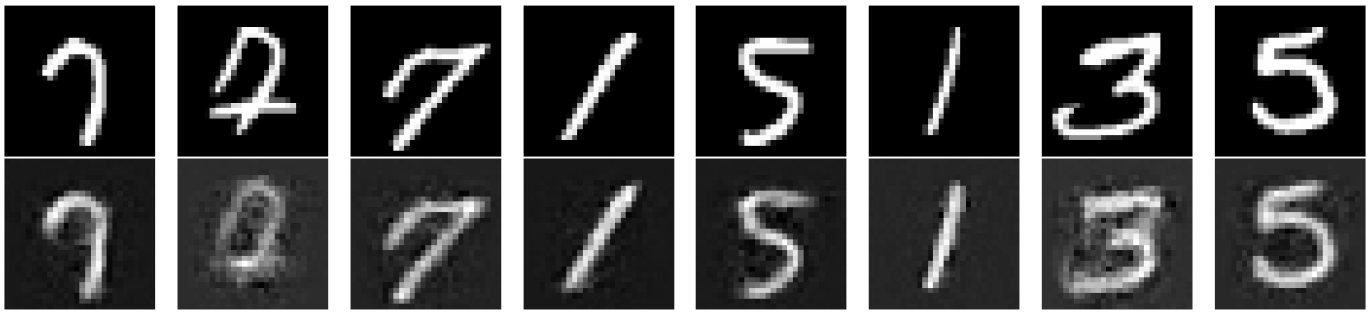
DECODED all-layers
ssim 0.34



④ Why the inverter matters — the WRONG one (naive SVD)

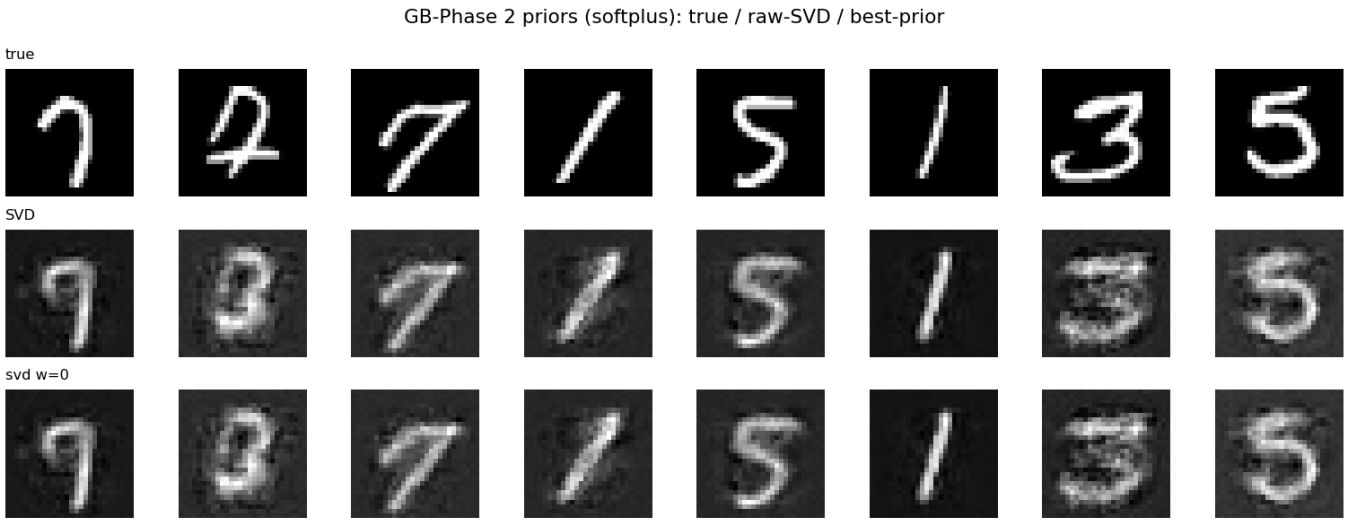
Experiment: input-layer SVD inversion (phase2_image.py). This takes the SAME decoded gradient as above but inverts it with plain linear algebra (SVD) instead of the model. Result: a coarse blob, SSIM ~0.17. This is the corner the attack was stuck in before we switched inverters. Compare row-for-row with pages ②/③.

GB-Phase 2 (softplus): decoded-gradient -> image (top=true, bottom=recovered)



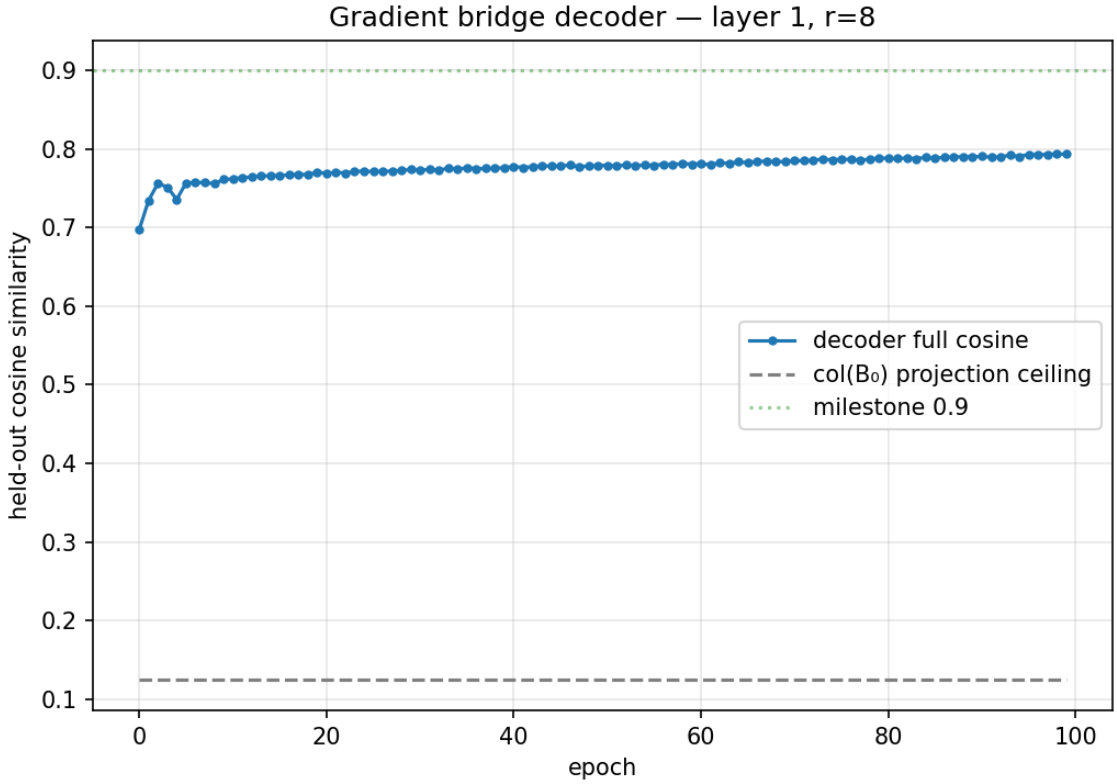
⑤ A dead end we ruled out — hand-crafted priors

Experiment: prior sweep (phase2_prior.py). We tried forcing the SVD image to be smooth (TV) or sparse (L1) — the usual tricks. Left col = truth, middle = raw SVD, right = best prior. Every prior made it WORSE, not better (see the STATUS note). That is what told us the problem was the inverter, not a missing prior — leading to the model-based fix on pages ②/③.



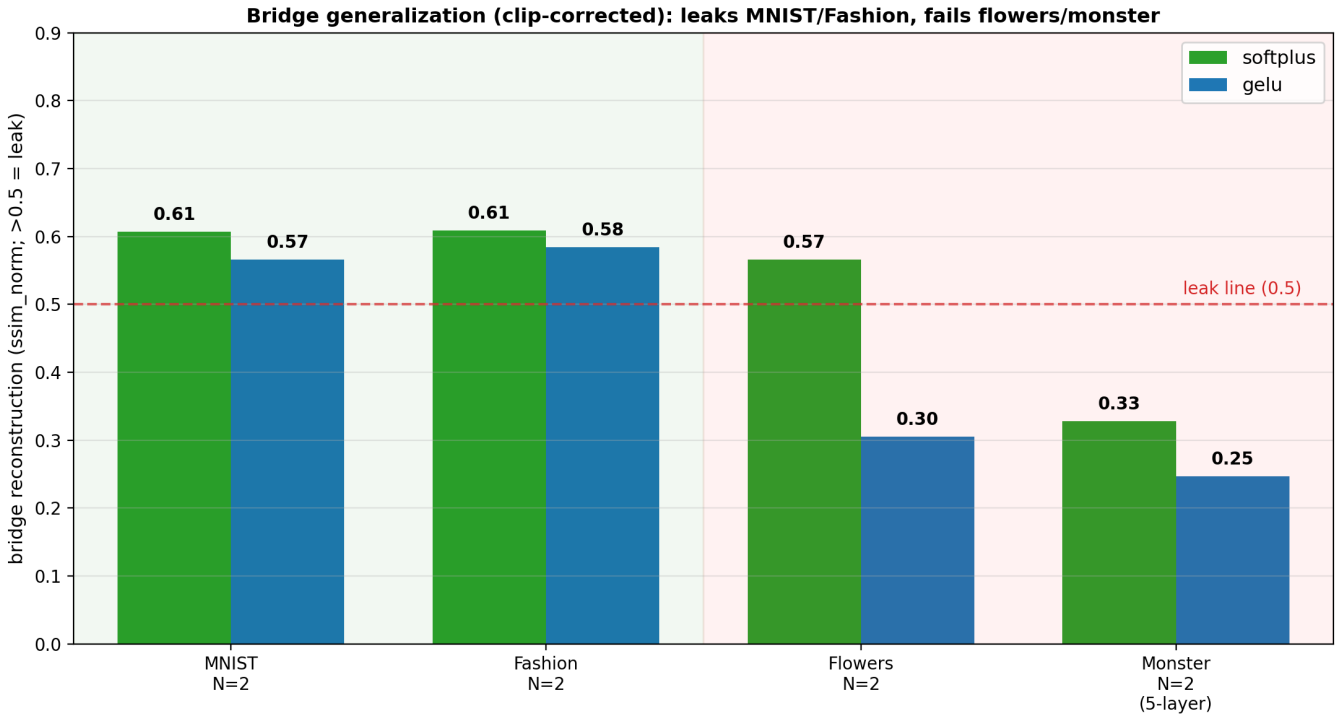
⑥ Under the hood — the decoder recovers the gradient well

Experiment: gradient-bridge decoder training (train_decoder.py). The decoder learns to rebuild the full-layer gradient from the tiny LoRA adapter. The curve climbs to ~0.99 cosine on held-out public data (this is the hidden layer). So the gradient recovery is strong — the remaining challenge is purely the inversion-to-pixels step.



⑦ Generalization scoreboard — flowers & more images

The bridge attack (adapter only) across data + number of images, one bar per activation. Green zone = MNIST, orange = flowers. Above the red line (0.5) = the private image leaked. Reads: leaks on MNIST at N=2, fades as N grows (superposition), and mostly STARVES on flowers (too little public proxy for 3072-dim RGB). Experiment: jobs 807059 (MNIST) + 949874 (flowers).



⑧ Flowers — softplus (marginal leak)

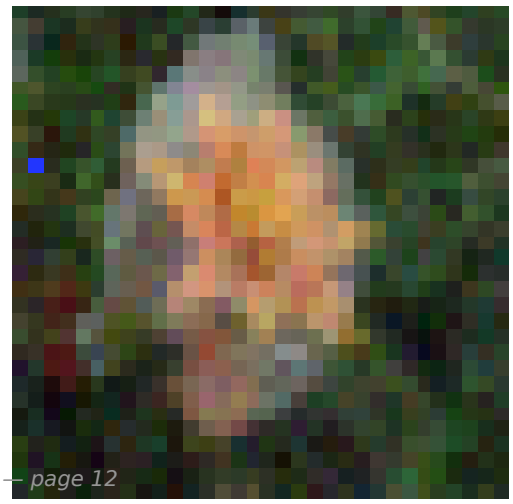
Experiment: phase2_e2e.py dataset flowers32. Top row = true private flowers, others = reconstructions. The DECODED (adapter-only) rows are washed out — ssim_norm 0.57, just over the leak line — while TRUE (full weight change) is a clean 0.75. The gap IS the bridge's data hunger: only ~7k public flowers to train a 134M-param decoder at 3072 dims.

(flowers32_N2_softplus): decoded ΔW ->

true

TRUE ΔW (ceiling)
ssim 0.76

DECODED all-layers
ssim 0.56



⑨ Flowers — gelu (bridge fails, direct is near-perfect)

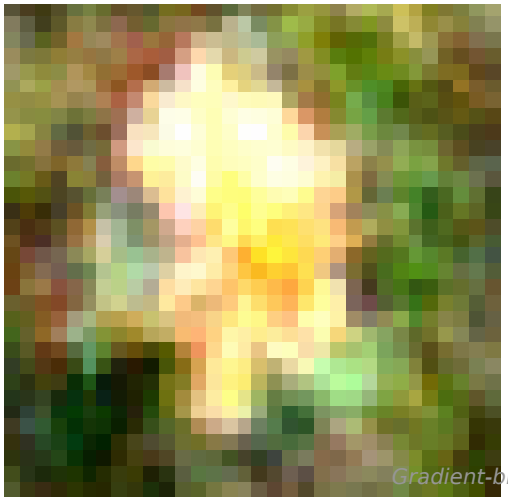
Same experiment, gelu. TRUE (full ΔW) reconstructs almost perfectly (ssim 0.999) but the DECODED bridge collapses (0.26) — the largest direct-vs-bridge gap we see. So on flowers the leak is real for a full-gradient attacker but NOT for an adapter-only attacker with scarce public data.

d (flowers32_N2_gelu): decoded ΔW -> n

true

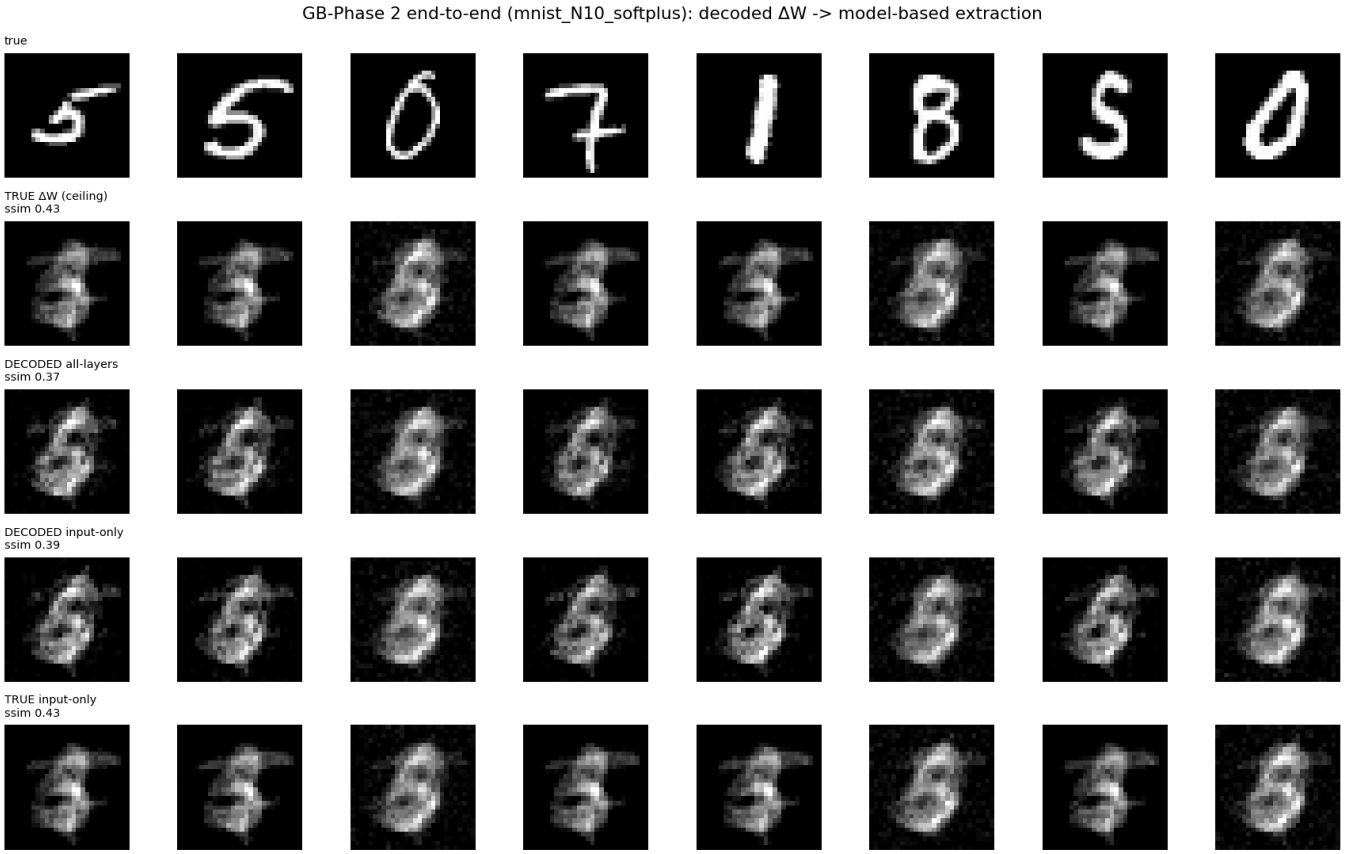
TRUE ΔW (ceiling)
ssim 1.00

DECODED all-layers
ssim 0.26



⑩ More than 2 images — the superposition wall (MNIST, N=10)

Experiment: phase2_e2e.py npc_list (N=10). With many images the per-image gradients blend, and BOTH the bridge and the full-ΔW attack degrade toward the mean-baseline. The bridge (DECODED) still tracks the direct attack (TRUE) — the adapter is not the extra bottleneck here; N is.



①① Bigger network (monster) — softplus, CIFAR-10 5-layer net

Experiment: phase2_e2e.py dataset cifar10 on a 19M-param, 5-layer net (job 997876). Top = true CIFAR images. TRUE (full weight change) reconstructs well (ssim 0.77; gelu hit 1.00) and all 5 decoders trained fine (0.86-0.96) — but the DECODED (adapter-only) bridge FAILS (ssim 0.30, below the 0.62 baseline). On this deep net, using ALL layers is WORSE than the input layer alone: stacking 5 imperfectly-decoded layers compounds error across depth. So the bridge does not scale to network depth, even though direct inversion does.

l (cifar10_N2_softplus): decoded ΔW -> n

true

TRUE ΔW (ceiling)
ssim 0.77

DECODED all-layers
ssim 0.31